

گزارش کار پروژه سوم آزمایشگاه سیستم عامل

زمان بندی پردازها، زمان بند چندصفی، WRR، SJF، FCFS و aging

فهرست

2	خلاصه اجرایی
3	فایل های اصلی تغییر داده شده
3	پاسخ پرسش های مفهومی پرداز و scheduler
3	۱. بررسی PCB و های state پرداز در xv6
3	۲. معادل وضعیت های xv6 در چرخه حیات پرداز
4	۳. گذار new به ready در xv6
4	۴. سقف تعداد پردازها در xv6
4	۵. چرا scheduler باید جدول پردازها را قفل کند؟
4	۶. ثبات Counter Program در context
4	۷. چرا ابتدای scheduler از sti استفاده می شود؟
5	۸. وقفه timer هر چند وقت یک بار رخ می دهد؟
5	۹. گذار interrupt در چرخه حیات پرداز
5	۱۰. بررسی quantum زمان بندی round-robin در xv6
5	۱۱. تابع wait در نهایت از چه تابعی برای منتظر ماندن استفاده می کند؟
5	۱۲. استفاده های دیگر sleep
5	۱۳. تابع sleep چه گذار وضعیتی ایجاد می کند؟
5	۱۴. برخورد xv6 با پرداز orphan
6	طراحی زمان بند چندصفی
6	صف ها و ثابت ها
6	فیلدهای اضافه شده به proc struct
6	الگوریتم های داخلی صف ها
6	صف صفر: Robin Round
6	صف یک: SJF تقریبی
7	صف دو: FCFS
7	انتخاب بین صف ها با WRR
7	سازوکار aging و جلوگیری از starvation
7	تغییرات لازم در exec و رفتار shell
8	فراخوانی های سیستمی اضافه شده برای زمان بندی
8	set_scheduling_info
8	change_queue
8	print_scheduling_info
8	پاسخ پرسش های مربوط به زمان بند جدید

8	۱۵. با CPU=2 آیا ترتیب قبلی حفظ می‌شود؟
8	۱۶. اثر سیاست‌های FCFS SJF و time-slice روی starvation
9	۱۷. چرا زمان SLEEPING waiting حساب نمی‌شود؟
9	برنامه‌های تست زمان‌بندی
9	schedtest
9	workload_short
9	workload_long
9	workload_mixed
9	workload_aging
10	مقایسه الگوریتم‌ها برای workloadها
10	راهنمای build و اجرای پروژه
10	اعتبارسنجی نهایی و سلامت تحویل
11	جمع‌بندی

خلاصه اجرایی

در پروژه سوم تمرکز روی زمان‌بندی پردازنده‌ها در xv6 بود. در حالت اولیه یک scheduler ساده دارد که پردازنده‌های RUNNABLE را به شکل round-robin انتخاب می‌کند. هدف این پروژه این بود که ابتدا چرخه حیات پردازنده و ساختار PCB در xv6 را بشناسیم و سپس scheduler را به یک زمان‌بند چندصفحه تبدیل کنیم.

در پیاده‌سازی این پروژه سه صف اصلی در نظر گرفته شد:

1. صف صفر: Robin Round با quantum پنج tick!
2. صف یک: SJF تقریبی با time burst و confidence!
3. صف دو: FCFS بر اساس زمان ورود به صف.

انتخاب بین صف‌ها با Robin Round Weighted انجام می‌شود تا هیچ صفی کاملاً حذف نشود. همچنین برای جلوگیری از starvation مکانیزم aging اضافه شد تا پردازنده‌هایی که مدت زیادی RUNNABLE مانده‌اند به صف بالاتر منتقل شوند.

[!NOTE] در این گزارش زمان‌بند جدید به عنوان یک توسعه روی scheduler ساده xv6 توضیح داده شده است؛ یعنی ابتدا رفتار پیش‌فرض xv6 بررسی می‌شود و بعد نشان داده می‌شود که هر فیلد و هر syscall جدید چه نقشی در تصمیم زمان‌بندی دارد.

نمودار زیر ساختار کلی صف‌های زمان‌بند چندصفحه را نشان می‌دهد:

flowchart TD

```

Timer[وقفه timer] --> Tick[تیک حساب‌داری tick]
Tick --> WRR[Weighted Round Robin با صف انتخاب]
WRR --> Q0[صف: صف: Round Robin]
WRR --> Q1[صف: یک: SJF تقریبی]
WRR --> Q2[صف: دو: FCFS]
Q0 --> Run[منتخب پردازنده اجرای Run]
Q1 --> Run
Q2 --> Run
Aging[aging] --> Q0[منتظر]

```

برای تخمین زمان اجرای بعدی در ایده SJF می‌توان از یک وزن اعتماد استفاده کرد:

$$B_{new} = \alpha B_{given} + (1 - \alpha) B_{old}$$

فایل‌های اصلی تغییر داده شده

فایل	نقش
proc.h proc.c	اضافه شدن فیلدهای زمان‌بندی به proc struct و cpu struct پیاده‌سازی SJF RR، تقریبی، FCFS، aging WRR و های syscall کمکی scheduler
trap.c exec.c	فراخوانی های scheduler tick هنگام وقفه timer تنظیم صف پیش‌فرض پردازش‌ها بعد از exec و حفظ در صف مناسب
sysproc.c ,user.h, syscall.c, syscall.h usys.S schedtest.c	پیاده‌سازی های kernel wrapper برای های syscall زمان‌بندی ثبت های syscall جدید
,workload_short.c ,workload_long.c ,workload_mixed.c workload_aging.c	برنامه تست عمومی scheduler برنامه‌های ارزیابی رفتار زمان‌بند و نمایش aging

پاسخ پرسش‌های مفهومی پردازش و scheduler

۱. بررسی PCB و های state پردازش در xv6

در xv6 ساختار PCB همان proc struct در فایل proc.h است. چند نمونه از فیلدهای آن و معادل مفهومی آن‌ها در مدل‌های رایج سیستم‌عامل:

مفهوم در PCB	فیلد در xv6	توضیح
شناسه پردازش	pid	شماره یکتای پردازش
وضعیت پردازش	state	یکی از وضعیت‌های UNUSED تا ZOMBIE
اطلاعات حافظه	pgdir, sz	اندازه حافظه و table page
متن اجرایی CPU	tf, context	context برای swtch و trapframe برای trap/syscall
والد پردازش	parent	رابطه درختی پردازش‌ها
فایل‌های باز	ofile[]	منابع فایل متعلق به پردازش
دایرکتوری جاری	cwd	مسیر کاری پردازش
نام پردازش	name	کمک به debug و نمایش

وضعیت‌های تعریف شده در xv6 عبارت‌اند از:

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

۲. معادل وضعیت‌های xv6 در چرخه حیات پردازش

وضعیت xv6	معادل مفهومی در چرخه حیات
UNUSED	خانه خالی، هنوز پردازش‌ای وجود ندارد
EMBRYO	new / در حال ساخته شدن
RUNNABLE	ready / آماده اجرا
RUNNING	running / در حال اجرا روی CPU
SLEEPING	blocked یا waiting / منتظر رخداد
ZOMBIE	terminated / پایان یافته ولی هنوز جمع‌آوری نشده

۳. گذار new به ready در xv6

برای ساخت پردازش جدید، تابع allocproc یک خانه آزاد از جدول پردازش‌ها پیدا می‌کند و state را از UNUSED به EMBRYO تغییر می‌دهد. سپس در userinit برای اولین پردازش و در fork برای پردازش‌های بعدی، پس از کامل شدن مقداردهی، state به RUNNABLE تغییر می‌کند.

پس گذار مفهومی new به ready در xv6 به صورت زیر است:

UNUSED -> EMBRYO -> RUNNABLE

تابع‌های مهم در این مسیر allocproc, userinit و fork هستند.

۴. سقف تعداد پردازش‌ها در xv6

حداکثر تعداد پردازش‌ها در param.h با NPROC مشخص شده است:

```
#define NPROC 64
```

اگر یک پردازش بیش از این تعداد child بسازد، allocproc خانه آزاد پیدا نمی‌کند و fork مقدار 1- برمی‌گرداند. kernel در حالت عادی panic نمی‌کند؛ برنامه user خطای fork را از مقدار برگشتی می‌فهمد.

۵. چرا scheduler باید جدول پردازش‌ها را قفل کند؟

جدول پردازش‌ها یک ساختار مشترک است. در سیستم چندپردازنده‌ای، ممکن است دو CPU هم‌زمان scheduler خود را اجرا کنند یا یکی در حال fork/exit/sleep/wakeup باشد و دیگری در حال انتخاب پردازش. اگر ptable.lock نباشد، دو CPU می‌توانند یک پردازش را هم‌زمان انتخاب کنند یا هال‌ها سازگار شوند.

حتی در سیستم تک‌پردازنده نیز وقفه‌ها و مسیرهای مختلف kernel می‌توانند state پردازش را تغییر دهند. بنابراین قفل کردن جدول پردازش‌ها هم برای SMP و هم برای حفظ های kernel invariant لازم است، هرچند در تک‌پردازنده بعضی‌ها race کمتر رخ می‌دهند.

۶. ثبات Counter Program در context

در ساختار context، معادل counter program ثبات eip است:

```
struct context {
    uint edi;
    uint esi;
    uint ebx;
    uint ebp;
    uint eip;
};
```

در switch مقدار eip به صورت مستقیم با دستور خاص ذخیره نمی‌شود؛ بلکه آدرس بازگشت تابع روی stack وجود دارد. وقتی esp ذخیره می‌شود، stack layout طوری است که این آدرس بازگشت در جای فیلد eip ساختار context قرار می‌گیرد. هنگام بازگرداندن، ret context باعث می‌شود اجرا از همان eip ادامه پیدا کند.

۷. چرا ابتدای scheduler از sti استفاده می‌شود؟

scheduler در ابتدای حلقه interrupts را فعال می‌کند تا CPU هنگام بیکار بودن یا جست‌وجوی پردازش، وقفه‌های timer و device را دریافت کند. اگر وقفه‌ها غیرفعال بمانند، رخدادهایی مثل پایان I/O یا timer tick پردازش نمی‌شوند و ممکن است پردازش‌های sleeping بیدار نشوند یا سیستم عملاً متوقف شود.

۸. وقفه timer هر چند وقت یک بار رخ می‌دهد؟

در xv6 واحد اصلی زمان‌بندی tick است. در این پروژه طبق صورت پروژه، پنج tick معادل حدود ۵۰ میلی‌ثانیه در نظر گرفته شده است؛ بنابراین هر tick تقریباً ۱۰ میلی‌ثانیه است. مقدار دقیق wall-clock در QEMU ممکن است به تنظیمات شبیه‌ساز وابسته باشد، ولی scheduler بر اساس شمارنده ticks تصمیم می‌گیرد.

۹. گذار interrupt در چرخه حیات پردازش

وقتی وقفه timer رخ می‌دهد و پردازش فعلی در RUNNING state است، در trap.c تابع yield صدا زده می‌شود. state yield پردازش را به RUNNABLE تغییر می‌دهد و سپس با sched کنترل را به scheduler برمی‌گرداند. پس گذار مربوط به interrupt به صورت زیر است:

RUNNING -> RUNNABLE

۱۰. بررسی quantum زمان‌بندی round-robin در xv6

در xv6 اولیه، پردازش در هر interrupt timer می‌تواند yield کند؛ بنابراین quantum موثر تقریباً یک tick است. در پیاده‌سازی این پروژه برای صف RR، quantum برابر پنج tick تعریف شد:

```
#define SCHED_RR_QUANTUM_TICKS 5
```

با فرض ۱۰ میلی‌ثانیه برای هر tick، quantum صف RR حدود ۵۰ میلی‌ثانیه است.

۱۱. تابع wait در نهایت از چه تابعی برای منتظر ماندن استفاده می‌کند؟

در wait، اگر فرزند zombie وجود نداشته باشد ولی هنوز child زنده‌ای وجود داشته باشد، پردازش والد با تابع sleep می‌خوابد:

```
sleep(curproc, &ptable.lock);
```

وقتی child در exit تمام می‌شود، parent با wakeup1 بیدار می‌شود.

۱۲. استفاده‌های دیگر sleep

علاوه بر wait، از sleep در بخش‌های دیگری نیز استفاده می‌شود. مثال‌ها:

- در sys_sleep، پردازش روی ticks می‌خوابد تا زمان مورد نظر بگذرد.
- در pipe، اگر pipe خالی باشد reader می‌خوابد و اگر pipe پر باشد writer می‌خوابد.
- در بعضی مسیرهای I/O، پردازش تا آماده شدن رخداد مورد نظر sleep می‌شود.

۱۳. تابع sleep چه گذار وضعیتی ایجاد می‌کند؟

تابع sleep پردازش در حال اجرا را روی یک channel می‌خواباند. بنابراین گذار اصلی آن:

RUNNING -> SLEEPING

بعد از wakeup، اگر رخداد مورد نظر اتفاق افتاده باشد، پردازش دوباره RUNNABLE می‌شود.

۱۴. برخورد xv6 با orphan پردازش

اگر parent زودتر از child تمام شود، child یتیم یا orphan می‌شود. در xv6 در تابع exit فرزندان پردازش خارج شده را به initproc می‌سپارد. سپس init با wait پردازش‌های zombie را جمع‌آوری می‌کند. این کار باعث می‌شود zombieها در سیستم باقی نمانند.

طراحی زمان بند چند صفی

صف ها و ثابت ها

در proc.h سه صف تعریف شد:

```
#define SCHED_Q_RR      0
#define SCHED_Q_SJF    1
#define SCHED_Q_FCFS   2
#define SCHED_NQUEUE   3
```

همچنین quantum و aging به صورت ثابت تعریف شدند:

```
#define SCHED_RR_QUANTUM_TICKS 5
#define SCHED_WRR_UNIT_TICKS  10
#define SCHED_AGING_TICKS     800
```

فیلدهای اضافه شده به proc struct

برای اینکه scheduler بتواند تصمیم بگیرد، چند فیلد به PCB اضافه شد:

فیلد	نقش
sched_queue	شماره صف فعلی پردازش
sched_wait_ticks	تعداد تیک های tick که پردازش در حالت RUNNABLE منتظر مانده است
sched_burst_time	تخمین زمان اجرای پردازش برای SJF
sched_confidence	میزان اعتماد به تخمین burst
sched_consecutive_ticks	تعداد تیک های tick اجرای پشت سر هم
sched_arrival_tick	tick ورود پردازش به صف فعلی
sched_shell_path	علامت برای حفظ مسیر shell در صف مناسب

برای هر CPU نیز وضعیت WRR نگهداری می شود:

فیلد	نقش
sched_queue	صفی که CPU فعلا سهم آن را مصرف می کند
sched_queue_ticks	تعداد تیک های tick مصرف شده از سهم صف فعلی
sched_rr_next	اندیس شروع بعدی برای RR صف صفر

الگوریتم های داخلی صف ها

صف صفر: Robin Round

در صف صفر، پردازش های RUNNABLE به شکل round-robin انتخاب می شوند. برای اینکه هر بار از ابتدای ptable شروع نشود، در فیلد cpu struct sched_rr_next نگه داشته می شود. هر پردازش در این صف حداکثر پنج tick پشت سر هم اجرا می شود و سپس با yield به scheduler برمی گردد.

صف یک: SJF تقریبی

در سیاست SJF واقعی، دانستن زمان اجرای آینده پردازش نیاز است که در عمل ممکن نیست. بنابراین در این پروژه هر پردازش دو مقدار دارد:

- sched_burst_time: تخمین زمان اجرا؛

- sched_confidence: درصد اعتماد به تخمین.

ابتدا پردازها بر اساس burst کمتر، arrival قدیمی‌تر و سپس pid کمتر مرتب می‌شوند. سپس با استفاده از confidence، انتخاب پردازها کوتاه‌تر به صورت احتمالی انجام می‌شود. اگر confidence بالا باشد، احتمال انتخاب پردازها با burst کمتر بیشتر است. این روش تقریبی است و هدف آن نزدیک شدن به ایده SJF بدون نیاز به پیش‌گویی قطعی آینده است.

صف دو: FCFS

در صف FCFS، پردازهای انتخاب می‌شوند که زودتر وارد صف شده باشند. این زمان در sched_arrival_tick ذخیره می‌شود. اگر دو پردازها arrival یکسان داشته باشند، pid کمتر اولویت دارد تا رفتار deterministic شود.

انتخاب بین صف‌ها با WRR

اگر همیشه صف بالاتر اولویت مطلق داشته باشد، صف‌های پایین ممکن است دچار starvation شوند. برای همین انتخاب بین صف‌ها با Robin Round Weighted انجام شد. وزن‌ها به شکل زیر هستند:

سهم	داخلی الگوریتم	صف
۳ واحد	RR	۰
۲ واحد	تقریبی SJF	۱
۱ واحد	FCFS	۲

هر واحد ۱۰ tick است، پس صف‌ها به ترتیب ۳۰، ۲۰ و ۱۰ tick سهم می‌گیرند. اگر صف فعلی پردازها RUNNABLE نداشته باشد یا سهم آن تمام شود، CPU به صف بعدی می‌رود.

سازوکار aging و جلوگیری از starvation

در هر tick، تابع scheduler_waiting_tick پردازهای RUNNABLE را بررسی می‌کند. فقط پردازهای waiting حساب می‌شوند که واقعا آماده اجرا هستند ولی CPU نگرفته‌اند. اگر پردازهای در صف FCFS مدت زیادی منتظر بمانند، به SJF منتقل می‌شود. اگر در SJF هم مدت زیادی منتظر بمانند، به RR منتقل می‌شود:

FCFS -> SJF -> RR

دلیل اینکه زمان SLEEPING حساب نمی‌شود این است که پردازها sleeping آماده استفاده از CPU نیست؛ مثلاً منتظر I/O یا رخداد دیگری است. waiting از دید scheduler یعنی پردازها می‌توانسته اجرا شود ولی انتخاب نشده است. در نسخه نهایی، هر بار که aging فعال شود، kernel بعد از آزاد کردن ptable.lock پیام زیر را چاپ می‌کند تا هم الزام صورت پروژو رعایت شود و هم ترتیب قفل‌های console و table process و خطرناک نشود:

aging: pid <pid> promoted from queue <old> to queue <new>

تغییرات لازم در exec و رفتار shell

طبق صورت پروژو، پردازها اولیه و shell باید در صف اول باقی بمانند، ولی برنامه‌هایی که از shell با exec اجرا می‌شوند به صف پیش‌فرض منتقل شوند. برای این کار تابع scheduler_on_exec بعد از موفق شدن exec صدا زده می‌شود.

در exec.c ابتدا نام برنامه در curproc->name کپی می‌شود و سپس بعد از commit شدن image جدید، scheduler با همین نام تصمیم می‌گیرد:

```
safestrncpy(curproc->name, last, sizeof(curproc->name));
...
scheduler_on_exec(curproc->name);
```

اگر نام `init` یا `sh` باشد، پردازش در صف `RR` باقی می‌ماند. در غیر این صورت به صف پیش‌فرض `FCFS` منتقل می‌شود. این کار باعث می‌شود `shell` تعاملی `responsive` بماند، ولی برنامه‌های عادی به شکل کنترل‌شده وارد `scheduler` چندصفی شوند.

فراخوانی‌های سیستمی اضافه‌شده برای زمان‌بندی

سه `syscall` برای کنترل و مشاهده `scheduler` اضافه شد:

```
int set_scheduling_info(int pid, int burst, int confidence);
int change_queue(int pid, int q);
int print_scheduling_info(void);
```

set_scheduling_info

این `syscall` تخمین `burst` و `confidence` پردازش را تنظیم می‌کند. مقدار `burst` باید مثبت باشد و `confidence` باید بین صفر و صد باشد. این اطلاعات در صف `SJF` استفاده می‌شود.

change_queue

این `syscall` پردازش را به صف مشخص منتقل می‌کند و `sched_arrival_tick` را به `tick` فعلی تغییر می‌دهد. اگر صف نامعتبر باشد، مقدار 1- برمی‌گردد. همچنین برای حفظ پاسخ‌گویی سیستم، انتقال `init` و مسیر `shell` به صفی غیر از صف صفر رد می‌شود.

print_scheduling_info

این `syscall` جدول پردازش‌ها را با اطلاعات زمان‌بندی چاپ می‌کند. ستون‌های خروجی شامل نام، `pid`، `queue`، `state`، `wait`، `confidence`، `burst`، `consecutive`، `arrival` هستند. در نسخه نهایی ابتدا `snapshot` اطلاعات پردازش‌ها زیر `ptable.lock` گرفته می‌شود، سپس `lock` آزاد می‌شود و چاپ انجام می‌گیرد؛ بنابراین هنگام `cprintf`، قفل جدول پردازش‌ها نگه داشته نمی‌شود.

نمونه قالب خروجی:

name	pid	state	queue	wait	confidence	burst	consecutive	arrival
sh	2	SLEEPING	0	0	50	2	0	10
schedtest	4	RUNNING	2	0	50	2	1	120

پاسخ پرسش‌های مربوط به زمان‌بند جدید

۱۵. با `CPUS=2` آیا ترتیب قبلی حفظ می‌شود؟

خیر، در حالت دو `CPU` ترتیب دقیق اجرا مانند حالت تک‌پردازنده قابل تکرار نیست. در `CPUS=1` فقط یک `scheduler` فعال است و ترتیب اجرا ساده‌تر دیده می‌شود. اما در `CPUS=2` هر `CPU` `scheduler` خودش را دارد و هر دو از جدول مشترک پردازش‌ها پردازش انتخاب می‌کنند. قفل `ptable.lock` از `race` جلوگیری می‌کند، ولی `interleaving` دو `CPU` باعث می‌شود ترتیب چاپ خروجی و ترتیب اجرای دقیق پردازش‌ها تغییر کند.

۱۶. اثر سیاست‌های `SJF`، `FCFS` و `time-slice` روی `starvation`

در `SJF` خالص، پردازش‌های طولانی ممکن است پشت پردازش‌های کوتاه زیاد منتظر بمانند. در `FCFS`، `starvation` داخل همان صف کمتر است، چون ترتیب ورود رعایت می‌شود، اما اگر صف‌های بالاتر همیشه مشغول باشند ممکن است صف پایین دیرتر `CPU` بگیرد. در `RR` همه پردازش‌های صف به شکل نوبتی `CPU` می‌گیرند و برای کارهای تعاملی مناسب‌تر است.

در پیاده‌سازی این پروژ، WRR بین صف‌ها و aging دو ابزار اصلی برای کاهش starvation هستند. WRR تضمین می‌کند که صف‌های پایین نیز سهمی از CPU داشته باشند و aging باعث می‌شود پردازش‌های که مدت زیادی منتظر مانده، به صف بالاتر منتقل شود.

۱۷. چرا زمان SLEEPING waiting حساب نمی‌شود؟

پردازش sleeping آماده اجرا نیست و حتی اگر CPU آزاد باشد نمی‌تواند جلو برود، چون منتظر رخدادی مانند I/O، pipe یا timer است. اگر زمان sleeping را waiting حساب کنیم، پردازش‌های که بیشتر I/O انجام می‌دهد ممکن است بی‌دلیل به صف بالاتر ارتقا پیدا کند. بنابراین فقط زمان RUNNABLE بودن معیار انتظار زمان‌بندی قرار گرفت.

برنامه‌های تست زمان‌بندی

schedtest

این برنامه شش child می‌سازد و برای هر کدام، burst queue و confidence مشخص می‌کند. parent بعد از fork و قبل از شروع کار، child با pid برگشتی تنظیمات را اعمال می‌کند. برای جلوگیری از race بین parent و child از pipe استفاده شده است؛ child تا وقتی parent تنظیمات را انجام نداده شروع به مصرف CPU نمی‌کند.

```
$ schedtest
schedtest: creating 6 CPU-bound children
schedtest: q0=RR, q1=approx-SJF, q2=FCFS
...
```

این تست برای مشاهده هم‌زمان سه صف و خروجی print_scheduling_info استفاده شد. در نسخه نهایی چاپ خطوط شروع/پایان‌ها child از parent انجام می‌شود تا در CPUS=2 خروجی user-level داخل هم نرود.

workload_short

این برنامه چند پردازش سبک و کوتاه می‌سازد و آن‌ها را در صف RR قرار می‌دهد. انتظار می‌رود پردازش‌های کوتاه با RR پاسخ‌گویی خوبی داشته باشند و هیچ child مدت زیادی منتظر نماند.

```
$ workload_short
```

workload_long

این برنامه چند پردازش سنگین‌تر می‌سازد و آن‌ها را در صف FCFS قرار می‌دهد. هدف مشاهده رفتار FCFS برای کارهای طولانی‌تر است. در این حالت پردازش‌های که زودتر وارد صف شده باشند اول اجرا می‌شوند.

```
$ workload_long
```

workload_mixed

این برنامه ترکیبی از پردازش‌های کوتاه و طولانی را بین هر سه صف پخش می‌کند. هدف این تست مشاهده اثر WRR است؛ یعنی صف‌های بالاتر پاسخ‌گوتر باشند ولی صف‌های پایین هم کاملاً بی‌نصیب نمانند.

```
$ workload_mixed
```

workload_aging

این برنامه برای نمایش مستقیم aging اضافه شد. یک child طولانی در صف FCFS قرار می‌گیرد و چند دیگر در همان صف منتظر می‌مانند. در حالت CPUS=1 بعد از حدود ۸۰۰ tick باید پیام promotion از طرف kernel چاپ شود.

```
$ workload_aging
aging: pid 12 promoted from queue 2 to queue 1
```

مقایسه الگوریتم‌ها برای workloadها

workload	رفتار مناسب‌تر	دلیل
کوتاه و تعاملی	RR	quantum کوتاه باعث پاسخ‌گویی بهتر و تقسیم عادلانه CPU می‌شود
محاسباتی کوتاه با burst قابل تخمین	SJF تقریبی	پردازه‌های کوتاه‌تر زودتر تمام می‌شوند و میانگین زمان انتظار کم می‌شود
کارهای طولانی و ترتیبی	FCFS	پیاده‌سازی ساده و رعایت ترتیب ورود، مناسب برای jobهای batch ساده
ترکیبی	MFQ/WRR همراه aging	بین پاسخ‌گویی، throughput و جلوگیری از starvation تعادل ایجاد می‌کند

در عمل، هیچ الگوریتمی برای همه شرایط بهترین نیست. به همین دلیل استفاده از چند صف و ترکیب سیاست‌ها منطقی است: RR برای SJF responsiveness، برای کاهش زمان انتظار کارهای کوتاه، FCFS برای ترتیب ساده، و aging برای جلوگیری از گرسنگی پردازنده‌ها.

راهنمای build و اجرای پروژه

build: برای

```
make clean
make
make fs.img
```

برای اجرای تک‌پردازنده و مشاهده ترتیب ساده‌تر:

```
make qemu-nox CPUS=1
```

برای اجرای دوپردازنده‌ای:

```
make qemu-nox CPUS=2
```

برنامه‌های تست داخل xv6:

```
schedtest
workload_short
workload_long
workload_mixed
workload_aging
```

اعتبارسنجی نهایی و سلامت تحویل

در بازبینی نهایی، علاوه بر build عادی، مسیر make dist و build مجدد از dist نیز بررسی شد؛ چون در پروژه‌های xv6، ناقص بودن EXTRA در Makefile یکی از خطاهای رایج تحویل است.

نتیجه	مورد بررسی
موفق	clean make
موفق	make
موفق	fs.img make
موفق	dist make
موفق	fs.img make && dist cd
موفق	make && dist cd
موفق	CPUS=4 اجرای dry-run
تولید topology صریح برای QEMU با smp - cpus=4, cores=1, threads=1, sockets=4	

اصلاحات نهایی بازبینی:

1. در Makefile، برنامه‌های workload_mixed.c, workload_long.c, workload_short.c, schedtest.c و workload_aging.c به EXTRA اضافه شدند تا خروجی dist بعد از clean make هم بتواند همه UPROGS‌های زمان‌بندی را بسازد.
2. picirq.c نیز به EXTRA اضافه شد تا kernel build در dist روی toolchain استاندارد کامل بماند.
3. در مسیر wakeup و kill، وقتی پردازش از SLEEPING به RUNNABLE برمی‌گردد، metadata زمان‌بندی شامل sched_wait_ticks, sched_consecutive_ticks و sched_arrival_tick تازه‌سازی می‌شود. این کار برای FCFS، tie-break در SJF و aging لازم است.
4. تابع print_scheduling_info به روش snapshot-then-print اصلاح شد تا ptable، lock هنگام چاپ روی console نگه داشته نشود.
5. aging اکنون پیام الزامی pid و صف مبدأ/مقصد را چاپ می‌کند.
6. تست‌های user-level در حالت CPUS=2 خروجی تمیزتری دارند، چون childها دیگر مستقیم روی console چاپ نمی‌کنند.

برنامه‌های تست اصلی داخل xv6:

```
schedtest
workload_short
workload_long
workload_mixed
workload_aging
```

جمع‌بندی

در این پروژه ابتدا ساختار PCB‌های state پردازش، sleep/wakeup، yield wait و timer trap در xv6 بررسی شد. سپس scheduler ساده xv6 به یک زمان‌بند چندصفی تبدیل شد. سه سیاست SJF RR، تقریبی و FCFS در صف‌های جدا پیاده‌سازی شدند و انتخاب بین صف‌ها با WRR انجام شد. برای جلوگیری از aging starvation اضافه شد. همچنین هابی syscall برای تنظیم و مشاهده اطلاعات زمان‌بندی نوشته شد و چند workload برای بررسی رفتار scheduler در حالت‌های مختلف طراحی شد. نتیجه این است که xv6 از یک scheduler آموزشی ساده به نمونه‌ای قابل مشاهده از زمان‌بندی چندصفی تبدیل شد.